



**Tecnológico  
de Monterrey**

# **06. Taller de Vibe-coding**

**Ciencia de Datos II**

**Jorge  
Juvenal  
Campos Ferreira**

✉ [juvenal.campos@tec.mx](mailto:juvenal.campos@tec.mx)

# Vibe Coding



El ★**vibe coding** es programar **describiendo la intención en lenguaje natural y dejar que un agente de IA escriba el código.**

El humano conduce, evalúa y corrige; no teclea syntaxis.

El nombre captura la esencia: estás "*vibrando*" con la máquina, comunicándole tu visión general y dejándola manejar la programación.

Es como ser el director de una película: tú defines la visión y la IA (el equipo de producción) ejecuta los detalles técnicos.



# Si la IA puede escribir código por mí, ¿Vale la pena entonces aprender a programar?

Si.

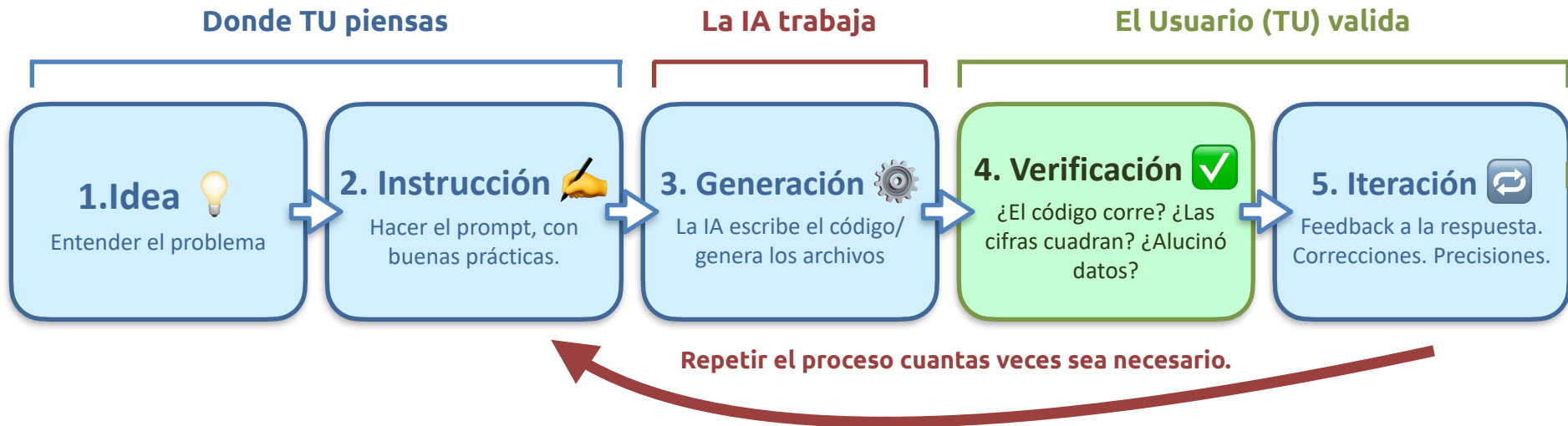
Antes aprender a programar era **la única forma** de crear software y hacer análisis de datos de forma eficiente y a escala. Aprender tomaba meses, dedicación y práctica.

Hoy también sirve para dirigir, evaluar y corregir lo que la IA produce. Sin fundamentos, las herramientas de IA son cajas negras por las que pasan cosas que no entendemos, y donde si algo sale mal, no sabemos por donde empezar a arreglarlo. Con **fundamentos**, estas mismas herramientas se vuelven **amplificadores**.

Además, aprender a programar permite:

- **Comunicar** con precisión los problemas que se quieren resolver
- **Disecionar** en partes, segmentos y módulos dichos problemas
- **Comprender** conceptos, herramientas y etiquetas técnicas: palabras y términos que te permiten pensar cosas que antes no podíamos pensar, pedir cosas que antes no podíamos pedir y entender y leer cosas que antes no podíamos entender.

# Los pasos del vibecoding



## Notas:

- 1) **La verificación es lo que separa al vibe-coding de un acto de fe.** Hay que tener forma de verificar que los datos que se generan/análisis que se hacen estén correctos.
- 2) **Cuidar el tamaño de la ventana de contexto:** llenarla consume más tokens y provoca más errores y alucinaciones
- 3) **Usar el modelo adecuado para la tarea adecuada.** Modelos más potentes pueden ayudar para la planeación; modelos más ligeros para las tareas sencillas.

# Prompting efectivo en vibecoding

## Elementos recomendados en un prompt de **vibecoding** (retomando de los vistos la semana pasada):

### Tarea o meta

Resultado a esperar.  
Meta a cumplir.  
Trabajo a realizar



### Contexto

Archivos de tu carpeta de trabajo a utilizar, memoria de la sesión, conversaciones previas, ejemplos importantes, información que le proporciones al agente, etc.



### Restricciones

Uso de estándares, reglas, arquitectura, convenciones.  
Definición de características deseables.



### Ejemplos

Ejemplos del resultado o uso de **Anclas** o valores esperados en el resultado: "El saldo de 2024 debe rondar los 171 mil; si el resultado no coincide algo está mal, ya sea el dato, el código o tu supuesto."



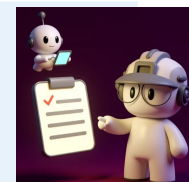
### Criterio de éxito

Condición verificable que señala que terminó: test que se pasaron, *bugs* que ya no se reproducen, auditorías que se cumplen.



### Modo Plan:

El modo planeación también es importante: activar ante tareas complejas/confusas/que puedan dar pie a ambigüedades.



# Ejercicio práctico

1. Genere y guarde una skill para una de las siguientes tareas repetitivas:

1. **checa-mi-entrega:** Una skill que tome una tarea, una rúbrica y le permita decirle el grado de cumplimiento de la tarea
2. **explica-mi-codigo:** Una skill que, tomando un código, le explique lo que hace de manera detallada.
3. **de-nota-a-guion:** Una skill que, tomando un texto, le genere un guión para un video corto de tiktok/reels/shorts
4. **de-material-a-guia:** Una skill que, tomando en cuenta material de clases, le genere una guía de estudio

2. En RStudio, instale las librerías DBI, RSQLite y dbplyr (y tidyverse, si no la tiene). Con los datos de la carpeta del *ejercicio\_02*, genere las consultas SQL que le permitan obtener de la base de datos *mr\_rib\_eye.db* las respuestas a las siguientes preguntas:

1. ¿Qué información tiene la base de datos?
2. ¿Cuáles son las diez ventas de mayor monto y en qué fecha ocurrieron?
3. ¿Cómo se reparten las 88 ventas entre efectivo, tarjeta y transferencia, tanto en número de tickets como en pesos? (Con gráfica)
4. ¿Qué sucursal genera más ingresos y cuál tiene el ticket promedio más alto? Ojo, no son la misma respuesta:
5. ¿Las propinas son proporcionalmente más generosas cuando se paga con tarjeta que con efectivo?
6. ¿Cuánto ingreso genera cada mesero por peso de salario base que recibe? Es un indicador de productividad que obliga a unir ventas, empleados y nomina.

Finalmente, revise y ejecute el código generado.

## Ejercicio práctico

3. En la carpeta “ejercicio\_03” están los datos de las proyecciones de población estatales de CONAPO a 2070 y las proyecciones de población de CONAPO municipales a 2040. A partir de estos datos, y usando *vibe-coding* y R, realice lo siguiente:

- a) Una gráfica de facetas que le permita ver la proyección poblacional de cada estado a 2070, además de conocer el año del pico poblacional. Guarde la imagen.
- b) Una ***aplicación shiny*** que permita visualizar, seleccionando estado y municipio, la proyección de población a 2040 de un municipio seleccionado.

4. En la carpeta “ejercicio\_04” están los insumos suficientes para replicar una aplicación que permite revisar, a nivel municipal, datos de preferencia electoral. El profesor proyectará la aplicación terminada. Por favor, utilizando ***vibe-coding***, intente replicarla.

# Funciones del Agente

**/help:** función que permite ver el catálogo de comandos en cada CLI

**/model:** permite ver y cambiar el modelo de IA que se está utilizando

**@archivo\_importante.R** Arrobar archivos cuando nos queremos referir a ellos le permite al agente encontrarlos de forma más sencilla e inequívoca, además de cargar solo la información del archivo que se necesita.

**/plan** Activa el modo planeación

**/permissions** (Claude Code) - **/approvals** (Codex - Opencode) Para darle permisos al agente de hacer cosas sin verificarlo siempre contigo.

**ESC (la tecla)** - Para interrumpir al agente a media tarea. **Doble ESC** (Claude Code) para regresar a un mensaje anterior y re-escribirlo.

**/rewind** (Claude Code) y **/undo** (Codex) para revertir acciones hechas en un prompt

**/context** para revisar cuanta ventana de contexto ya tienen ocupada

**/clear** para borrar toda la ventana de contexto y comenzar una tarea nueva - **/compact** para resumir la ventana de contexto antes de que se desborde

**/cost** y **/usage** para ver cuanto va costando (en uso, en tokens o en dinero) lo que llevamos de la sesión

**/init** genera un archivo CLAUDE.md (Claude Code) o AGENTS.md (Codex) que funcionan como archivos de configuración - prompts del sistema del proyecto que voy a trabajar.

**#** - Guarda una instrucción directamente en la memoria.

**/memory** permite acceder a los archivos de memoria (del usuario o del proyecto)

**/export** exporta la conversación completa a un archivo o al portapapeles

**/btw** (Claude code) sirve para platicar con el agente mientras trabaja (sin interrumpirlo)

# Capacidades del Agente

- **Búsqueda web** — El agente puede buscar en internet para responder. Claude Code y OpenCode la traen integrada; en Codex hay que habilitarla en la configuración.
- **Pegar imágenes (multimodalidad)** — Puedes arrastrar una imagen a la terminal o pegarla con Ctrl+V y el agente la ve.
- **Procesos en segundo plano** — El agente puede lanzar un proceso que sigue corriendo sin bloquear la conversación, y seguir editando código; luego puede leer la salida de ese proceso para diagnosticar errores.
- **Lista de tareas** — En tareas largas, Claude Code muestra una lista de pendientes que el agente va marcando conforme avanza. Sirve para auditar en qué va y detectar si se desvió del plan.

# Gracias

**¿Preguntas?**